

xMCF

ISO/PAS 8329

Description of mechanical connections
and joints in structural systems

ISO/PAS 8329 xMCF WG meeting
focus group on issues [#105](#) / [#61](#)
2026-03-25

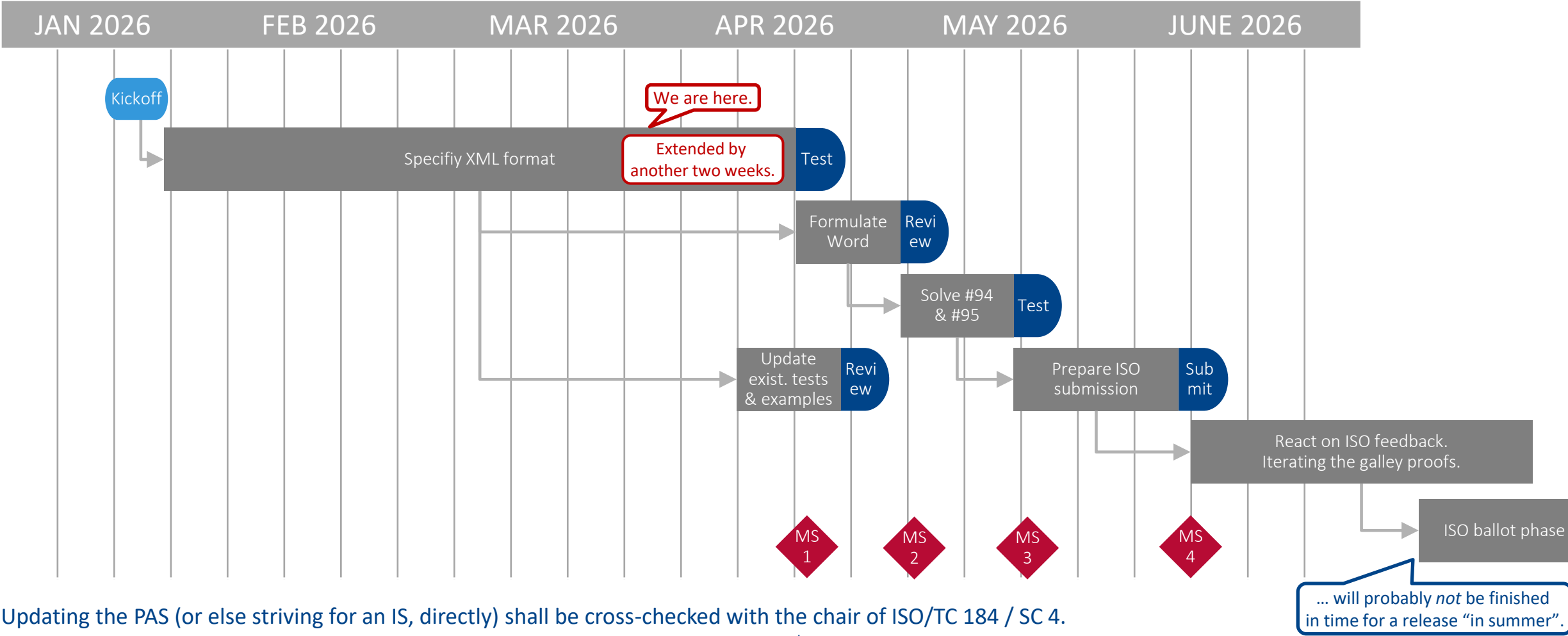
Agenda

As announced at https://github.com/economidis-nick/createXSDforxMCF/blob/master/VDA_FAT_AK_25/Meetings/2026-03-25_VideoConference/README.md

- Welcome & Flying over today's topics
- Timeline for next release
- Results of last PPC (if any)
- [Schematron](#)
- How do we design templates in the XSD file? — Nick and Stephan's [discussion at issue #105](#)
- Focus on detailing the XML elements, based at issue [#105](#)
- Steps we should take next



Timeline for the next release



Updating the PAS (or else striving for an IS, directly) shall be cross-checked with the chair of ISO/TC 184 / SC 4.
Recent meeting of the Policy and Planning Committee (PPC) was scheduled for 18th of Feb.



Leveraging Schematron XML verification

We discussed:

- Leveraging the “[power of Schematron](#)” for (which of) our purposes.
- Priority assigned to setting up this means of verification.

What are relevant scenarios / use cases?

- Developing export algorithms & verification of the results.
- Quality assurance when receiving data from (external) partners.
- Further development of the standard. Verification of examples. Development of recommended practices.
- Definition of different, increasingly strict rules along the product development process:
At the beginning of the PDP, χMCF may have many “gaps,”
but towards the end, many optional attributes should be provided.

Decision, 2026-03-11:

We will care for the XSD file, first, and then see, whether we need Schematron capabilities, on top.

We consider providing one or several Schematron files to the next or to a following publication of χMCF.

Nick and Stephan’s [discussion at issue #105](#) may shed a new light on using Schematron.
→ See later slide!

How do we design templates in the XSD file?

Stephan's [comment to issue #105](#)

Quote:

„It is not possible to define two different `<xs:element name="connection_0d"/>` inside the same scope. This means, we have to use the same `xs:element` for both, the `<connection_template/>` and a standard definition inside a `<connection_list/>`. This subsequently means that we have to remove anything from the `<connection_0d/>` that is mandatory, e.g. the `"connection_0d_type"`.“

BETA CAE suggested a solution. — See according [comment in issue 105 of the very same evening](#).

Issue is that mandatory attributes become optional, if a connection refers to a template, which provides the attribute.
How to represent this in XSD?

Sub-type declaration to be separated from “simple” attributes?

Assumption: XSD will not be able to help, once we introduce *external* templates.

To be verified: Can Schematron (or XSD) open an *external* template via e.g. an URL?

→ **Homework** / Thomas to involve Kevin.

Nick and Stephan's discussion at issue #105

Stephan [summarized the approach](#) as follows:

- Relax all relevant attributes from required to optional.
- Add "template" attributes inside the elements that were selected by the focus group.
- Add new elements for the templates themselves e.g. "washer_template".
- Use xs:assert to ensure that referenced templates exist.
- Use xs:assert to ensure that formerly required attributes are still present either in the template or at the position where the template is referenced.

However, some issues are left over. ⇒ Discussion:

- Nick's analysis is based on "paper documentation". Has not been verified by some example, yet.
- XML schema cannot verify a reference to an *external* template, contained in a file.
Schematron would be able to do this.
External template $\notin$ #105, but we should strive for a general solution which verifies both, internal and external templates.
- Next step: Verification of such an example by use of an XML tool. Joint effort by NE, GH & possibly KLT. Review by SH.
Gum Drop & Rivet examples from different sides and then exchange.

Detailing XML Elements — Template Syntax, Screw Example

We identified the example XML file [chapter 9 5 7 exampleA patched for xMCF3.2.xml](#) (provided by the comment of [2026-10-14 09:17+0100](#)) as a good starting point.

If yes, the template must *not* have an attribute “**label**”, since this would usually be *unique to each single* connector instantiated from the template.

It suggests as a syntax for defining an (file-embedded) list of *connection templates* at XML root level:

```
<connection_templates>
```

```
<connection_0d label="screw template 4711">
```

For `<washer/>` and we introduced the new attribute “`template_label`”. Shouldn’t we use it here, too? (I

Decision, 2026-03-11:
We change to „`template_label`” for all kind of templates.
→ Draft example files to be updated.

```
<threaded_connection part_code="M10x50 12.9" length="50." diameter="10"
```

```
head_diameter="16.0" head_height="5" sink_size="4" thread_length="35" >
```

```
<!-- Attributes "torque", "angle" and "pretension" must not appear in a template! -->
```

```
<screw /> <!-- Screw may come without any attributes -->
```

```
<washer outer_diameter="20"/>
```

```
</threaded_connection>
```

```
</connection_0d>
```

```
</connection_templates>
```

Nick and Stephan’s [discussion at issue #105](#) may shed a new light on attribute naming.
→ See according slide!

Homework: Consider what “misuse” or contradictory set of data can occur by this approach!

Detailing XML Elements — Reference Syntax, Screw Example

The example suggests as syntax for *referring to* a connection template at `<connection_0d/>` level:

Default form — no change to / overwriting template data:

```
<!-- Screw exactly as defined in connection_templates: -->
<connection_0d label="SCREW_100532" template="screw_template_4711">
  <threaded_connection>
    <normal_direction x="2.9" y="0.1" z="0.0" />
  </threaded_connection>
  <loc> 1500.3809 838.75885 730.6529 </loc>
</connection_0d>
```

Minimal change — overwriting an attribute value:

```
<!-- Screw exactly as defined in connection_templates, but with smaller torque: -->
<connection_0d label="SCREW_100533" template="screw_template_4711">
  <threaded_connection torque="70" >
    <normal_direction x="3.0" y="0.0" z="0.0" />
  </threaded_connection>
  <loc> 1540.3809 838.75885 730.6529 </loc>
</connection_0d>
```


Detailing XML Elements — Reference Syntax, Screw Example

The example suggests as syntax for *referring to* a connection template at `<connection_0d/>` level:

Overwriting a child element:

```
<!-- Screw exactly as defined in connection_templates, but with larger washer: -->
<connection_0d label="SCREW_101537" template="screw_template_4711">
  <threaded_connection>
    <normal_direction x="3.1" y="-0.1" z="0.0" />
    <washer outer_diameter="30"/>
  </threaded_connection>
  <loc> 1580.3809 838.75885 730.6529 </loc>
</connection_0d>
```

Attention: Here, only the `outer_diameter` of the `<washer/>` is overwritten. Any other washer-parameter given from the template stays effective.

Detailing XML Elements — Template Syntax, Bolt Example

The example extends to cases of *deeper structured* elements:

<connection_templates>

```
<connection_0d label="bolt_template_4712">
  <threaded_connection part_code="M10x50 12.9" length="50" diameter="10"
    head_diameter="16.0" head_height="5" sink_size="4" thread_length="35" >
    <!-- Attributes "torque", "angle" and "pretension" must not appear in a template! -->
    <normal_direction x="0" y="0" z="-10"/>
    <washer outer_diameter="20" inner_diameter="10.3" thickness="1.5" attached="true"/>
    <!-- Washer attached to the screw head -->
    <bolt>
      <nut diameter="16." height="5" />
      <!-- Friction attributes must not appear in a template,
        unless it is against an attached washer -->
      <!-- Attributes "clipped_to" or "fixed_to" must not appear in a template,
        since they each contain a flange partner index! -->
    </bolt>
  </threaded_connection>
</connection_0d>

```

</connection_templates>

For <washer/> and <nut/>, we introduced the new attribute "template_label". Shouldn't we use it here, too? (Instead of "label"). (Cf. slide 7.)

Nick and Stephan's [discussion at issue #105](#) may shed a new light on attribute naming.
→ See according slide!

Detailing XML Elements — Reference Syntax, Bolt Example

The connection template can be *referred to* for example like this:

```
<!-- Bolt exactly as defined in connection_templates, but with less torque, larger washer & nut: -->
```

```
<connection_0d label="BOLT_100732" template="bolt_template_4712">
```

```
  <threaded_connection torque="70">
```

```
    <normal_direction x="3.1" y="-0.1" z="0.0"/>
```

```
    <washer outer_diameter="30"/>
```

Check, whether a washer may refer to a connection template, too! → we need examples to better understand. → next slides!

```
    <bolt>
```

```
      <nut height="8"/>
```

Check, whether a nut may refer to a connection template, too! → we need examples to better understand. → next slides!

```
    </bolt>
```

```
  </threaded_connection>
```

```
</connection_0d>
```

Rule (→ Word file): If the connection template is a `<screw/>`, the referring element must not redefine it to a `<bolt/>` and vice versa!

To be extended to similar situations in case of future extensions of χMCF, e.g. a spotweld (from template) must not be redefined to a gumdrop!

Detailing XML Elements — Template Syntax for a Washer

Draft example:

```
<!-- The new parameter "template_label" is allowed for a template, only. -->
<washer template_label="template for a loose washer"
  outer_diameter="20" inner_diameter="10.3" thickness="1.5"
  strength_property_class="12.9" part_code="M10x20x1.5 12.9" />
<!-- Washer next to nut would have friction to last part, which is unknown. Thus, not allowed. -->
<!-- A washer can be attached to a screw head or a nut.
      But here as a template without context, one can't know. Thus, "attached" is not allowed. -->
```

Remarks:

- Templates must not contain friction parameters, because friction always is a physical property of a *combination* of two items.
 - We should differentiate between “firmly fixed” washers (i.e. washer and nut/screw form one solid) and “rotate-able” ones, where the washer and nut/screw can rotate independently around their common axis.
- Remark: BETA CAE’s study (2025-02-18) showed that we do *not* need a new attribute, here.

Detailing XML Elements — Template Syntax for a Nut w/ Washer

Draft example:

```
<!-- The new parameter "template_label" is allowed for a template, only. -->
<nut template_label="template for a nut with an attached washer"
  diameter="16." height="5" strength_property_class="12.9" part_code="M10x5 nut 12.9"
  static_friction="0.8" kinetic_friction="0.6" >
  <!-- Attributes "torque" and "angle" must not appear in a template! -->
  <!-- Attributes "clipped_to" or "fixed_to" must not appear in a template,
    since they each contain a flange partner index! -->
  <!-- The washer of this nut is attached to it in such a way that it can rotate.
    Therefore, the nut's friction against the washer is meaningful and allowed. -->

  <!-- If the washer is part of or firmly attached to the nut, it cannot have a part code! -->
  <washer outer_diameter="25" thickness="1.5" strength_property_class="12.9"
    attached="true" static_friction="0.7" kinetic_friction="0.5"/>
  <!-- The friction of the washer describes the contact to the connected part. -->
  <!-- This washer could reference a template, itself. -->
</nut>
```

Remark: Friction between nut and attached but rotate-able washer is allowed!

Detailing XML Elements — Template Syntax for a Bolt, using Washer & Nut templates, itself.

Draft example:

For `<washer/>` and `<nut/>`, we introduced the new attribute `"template_label"`. Shouldn't we use it here, too? (Instead of `"label"`). (Cf. slide 7.)

Nick and Stephan's [discussion at issue #105](#) may shed a new light on attribute naming. → See according slide!

```
<!-- The next bolt reference references washer and nut templates, itself: -->
<connection_0d label="bolt_template_4733">
  <threaded_connection length="50" diameter="10" head_diameter="16" head_height="5" thread_length="35"
    part_code="M10x50 12.9">
    <!-- Attributes "torque", "angle" and "pretension" must not appear in a template! -->
    <normal_direction x="0" y="0.8" z="-1.2"/>
    <washer template="template for a loose washer"/> <!-- The washer next to the bolt's head. -->
    <bolt>
      <nut template="template for a nut with an attached washer"/><!-- Nut has attached washer.-->
    </bolt>
  </threaded_connection>
</connection_0d>
```

Detailing XML Elements — Template Syntax, Gumdrops Example

(new — see file “chapter_10_5_exampleB__with_templates.xml”)

For a gumdrop, the example template would be:

```
<connection_templates>
  <connection_0d label="GumDrop template">
    <!-- Assumed Unit system with mass attribute with value="kg" -->
    <gumdrop diameter="5.0" mass="0.0033" material="Glue_Material" />
  </connection_0d>
</connection_templates>
```

For <washer/> and <nut/>, we introduced the new attribute “**template_label**”. Shouldn’t we use it here, too? (Instead of “**label**”). (Cf. slide 7.)

Nick and Stephan’s [discussion at issue #105](#) may shed a new light on attribute naming. → See according slide!

Detailing XML Elements — Reference Syntax, Gumdrops Sequence (new — see file “chapter_10_5_exampleB__with_templates.xml”)

For a gumdrop sequence, the example reference could be (as suggested in [GitHub comment](#)):

```
<!-- Sequence of gum drops exactly as defined in connection_templates: -->
<connection_1d label="DROP_LINE_11000" template="GumDrop template">
  <sequence_connection_0d spacing="30.0" margin="1.0" priority="spacing">
    <gumdrop/>
  </sequence_connection_0d>
  <loc_list>
    ...
  </loc_list>
</connection_1d>
```

Observation (CF): A template for a 0d item as parameter of a 1d item appears inappropriate to me.

What about this approach:

```
<connection_1d label="DROP_LINE_11000" template="Later, here could be place for a sequence template.">
  <sequence_connection_0d spacing="30.0" margin="1.0" priority="spacing" template="GumDrop template">
```

Decision, 2026-02-25:

`<sequence_connection_0d/>`
will *not* be considered within the scope
of our current work, just as has been
decided for all the other
`<connection_1d/>` types, before.

XML Example File provided by [@ghirel](#) / PDM-IF.

The example “SAMPLE-FASTENER_003--.--1In Work000-ACOMMON_ENG_01_xmcf3.2 GHi.xml” has shown us that:

- `<loc/>` must not appear inside `<connection_templates/>`.
- `<stacking/>` must not appear inside `<connection_templates/>`.



Contentual Action Items $\triangleq$ Next Steps:

- **Standard document (Word file):**

- Description of overall concept & objectives, intended application & motivation etc.
- Description of semantics for internal catalog
- Description of semantics for reference to a connection template
- Definition of example use cases and in-document XML-examples

- **XML & XSD** — See discussion in [#105!](#)

- Sketching XML example file(s), incl. (w.i.p.)
 - precedence rule (w.i.p.) and dropped mandatory attribute,
 - internal (w.i.p.) and external file-based catalog
- Description of syntax for internal catalogs (w.i.p.)
- Description of syntax for reference to a connection template:
 - internal (w.i.p.), external file-based, external database-based

- Drafting XML schema (XSD) and test cases / test example files

Homework: Collect use cases which can yield e.g. test cases, shared as Markdown files in GitHub at https://github.com/economidis-nick/createXSDforxMCF/tree/issues_61_94_95_105/VDA_FAT_AK_25/Meetings/2026-02-18_VideoConference

- Investigate xMCF elements and attributes for eligibility for template.
⇒ we need a rule, here! ⇒ Homework!



Next meetings

- 2026-04-01, 15:00 CEST — full group
 - objective: agree on XML format suggestions & examples
- 2026-04-08, 15:00 CEST — t.b.d.
 - objective: t.b.d.
- 2026-04-15, 15:00 CEST — t.b.d.
 - objective: t.b.d.
- 2026-04-22, 15:00 CEST — t.b.d.
 - objective: t.b.d.

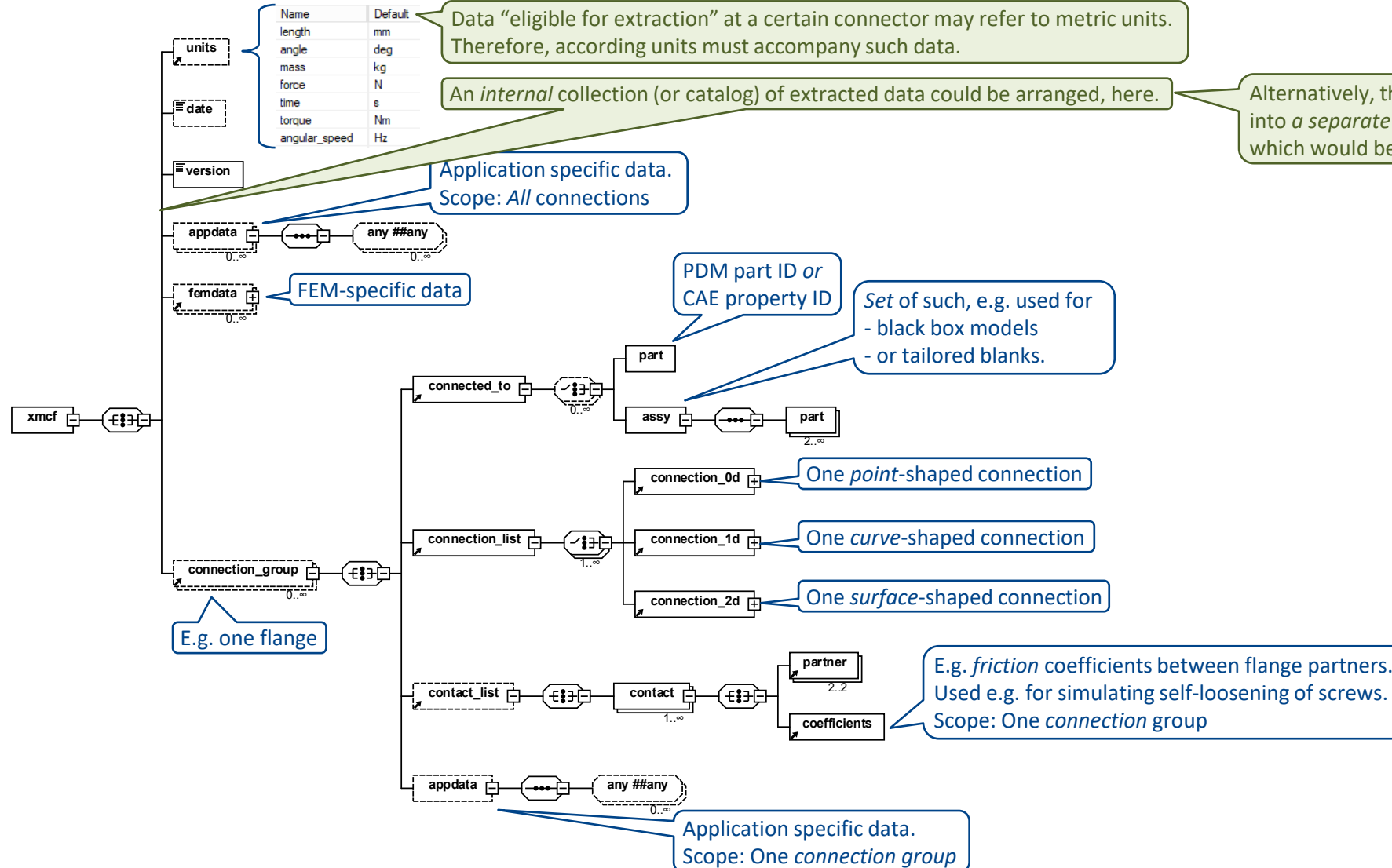
Rather the technical sub-group.

CF sent / will send out invitations.



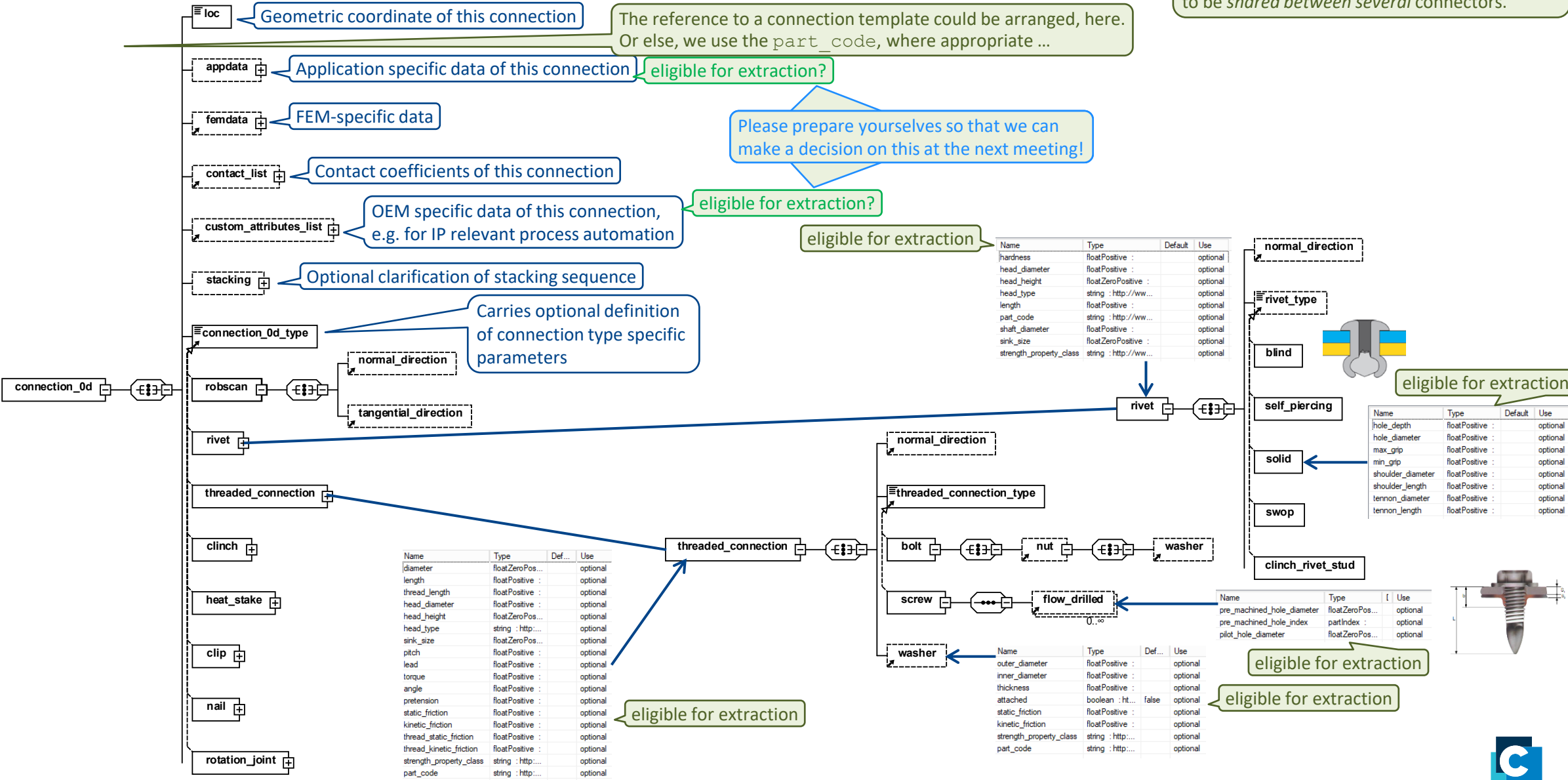
Structure of χ MCF 3.1.1 XSD – High Level

“eligible for extraction” to be understood as:
Data can be *taken out of one* specific connector,
to be *shared between several* connectors.



Structure of xMCF 3.1.1 XSD – Point Connections

“eligible for extraction” to be understood as:
Data can be *taken out of one* specific connector,
to be *shared between several* connectors.



Questions & Considerations of the 2025-11-26 Meeting



1. Q: What happens, if a connector contains both, a reference to extracted data (catalog) *and* (some of) such data, internally?
Which data takes precedence?
A: “Internal” data shall take precedence.
2. Q: May a connector omit *mandatory* attributes if they can be determined via the reference to a catalog entry?
A: Yes, but schema validation must take this into account.
3. Q: Should a catalog entry be allowed to refer to other entries, maybe even in another catalog?
Could be used to enable a *step-by-step completion*.
A: Not allowed, for internal catalogs, for the moment.
Format of database-catalogs is not subject of the standard, therefore no statement given here.
4. Q: Should we allow both, *internal & external* catalogs?
Even in combination?
A: Yes.

- Tools need to store, where data “eligible for extraction” originally was read from (individual connector, internal or external catalog).
- For new connectors, users must be able to specify and for existing ones, users must be able to modify this. In both cases, incl. the URL of the external catalog.
- Q: Should the system allow the modification of data originating from an external catalog?
A: Yes, but this creates internal data.
- Q: When writing an χ MCF file:
Should the system emit a warning if data in an *external* catalog has changed, meanwhile?
Or, if the external catalog can no longer be found. (To avoid lost or inconsistent data, in the end.)
A: References to external databases must consider life cycle (e.g., release codes) of the items contained therein.
However, we can leave this responsibility to the process owner. (Can be discussed further, later.)

Proposed wording for results of our previous meetings

(2025-11-26 & 2025-12-10)



- We will focus entirely on `connection_0d` and leave `connection_1d` and `connection_2d` aside for the moment.
However, we will take care not to impair any future expansion.
- We will allow both, *internal* & *external* catalogs, even in combination. — Where:
- An *internal* catalog resides *within* the specific χ MCF file at root level, as a sibling element of, for example, `<units/>`.
- An *external* catalog is located *outside* the specific χ MCF file. It can be a simple file, a collection of data records in a database or something else.
The standard should be formulated agnostically in this regard, with any reference to items in the external catalog described by a URL.

Proposal for continuation:

- If the *external* catalog is a simple file, its syntax should follow the already defined χ MCF syntax as closely as possible.
Future extensions to the basic syntax should, if possible, automatically yield the syntax extension for the external file.

Confirmation? ...

Proposed wording for results of our previous meetings (2025-11-26 & 2025-12-10)



- If a connector in a χ MCF file contains both,
 - a reference to catalog data,
 - and (some of) such data internally in its definition,the internal data shall take precedence.
- A connector may omit otherwise mandatory attributes if these can be determined via a reference to a catalog entry. XML schema validation must take this into account.
- Note (recursive references): A catalog entry *in χ MCF format* (either internal or external file-based) may *not* (for the time being) refer to other catalog entries itself. The format of *database-based* catalog entries is not subject to the standard. Therefore, no statement is made here.

At some point in the future, however, this feature could still prove useful for enabling the step-by-step completion of connector definitions.

Guillaume's questions of the 2025-11-26 meeting



- Catalogue, as identified using [VDA231-200](#) used at BMW: standard type (DIN EN, GS, SAE, ...), standard number, legal owner (especially important for internal company standards), issue date → [May be mapped to components of the URL?](#)
- `part_code`: At BMW, the welding points (spotwelds) are represented by a small sphere, which has a part number (see graphic) (not for logistic purposes, since it cannot be ordered or manufactured) → [We should transport this code.](#)
[The attribute name remains to be discussed, though.](#)
- Potential attributes dedicated to each *individual* usage of a fastener: location/orientation, *instance number* (see [#94](#)), *tightening torque*. → [To be considered!](#)
- Difference between `connection_[012]d.label` and `part_code`? → See issue [#104](#).
→ [Maybe, formulation in standard document needs improvement.](#)
[Please leave a hint in \[#104\]\(#\), in this case.](#)

